

Project Title	Enterprise-Grade Recommendation System with Deep Learning
Skills take away From This Project	Synthetic Data Generation, Data Modeling & Simulation, Recommendation Systems, Collaborative Filtering, Content-Based Filtering (NLP), Deep Learning with PyTorch, Neural Collaborative Filtering (NCF), Evaluation Metrics for Recommenders, System & Product Thinking, FastAPI / Streamlit Application Development, Python, Git, Documentation
Domain	E-Commerce / OTT Platforms / Online Learning / Retail / Media Streaming

Problem Statement:

You are working as a **Senior Data Scientist** at a digital platform.

The company is in an **early-stage or new-market scenario**, where:

- Real user interaction data is **limited or unavailable**
- Models must be **prototyped using synthetic data**
- System design must be **future-proof**

Your task is to build an **end-to-end Recommendation System** that:

1. **Generates realistic synthetic data**
2. Builds baseline and deep learning recommenders
3. Handles cold-start scenarios
4. Evaluates recommendation quality
5. Exposes recommendations via an application layer

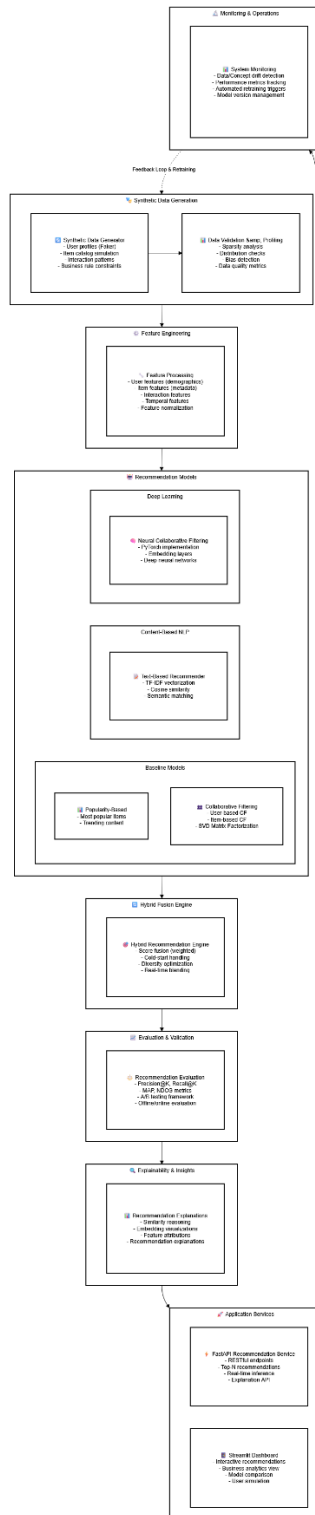
Business Use Cases

Modern digital platforms such as **e-commerce websites, OTT platforms, and online learning portals** rely heavily on recommendation systems to drive **user engagement, retention, and revenue**.

However, in many real-world scenarios—such as **new product launches, early-stage startups, privacy-restricted environments, or internal POCs**—**real user interaction data is either unavailable, incomplete, or highly sparse**. Despite this limitation, organizations are still expected to **design, prototype, and validate recommendation systems** before full-scale deployment.



ML Architecture Diagram



Approach

Synthetic Data Generation (CRITICAL PHASE)

Students must **programmatically generate** realistic recommendation data using Python.

Data to Generate

Users Table

```
user_id  
age  
gender  
location  
user_segment
```

Items Table

```
item_id  
category  
title  
description  
price  
content_tags
```

Interactions Table

```
user_id item_id rating / click / watch_time / purchase timestamp
```

Synthetic Data Requirements

- Minimum:
 - 5,000 users
 - 2,000 items
 - 100,000+ interactions
- Introduce:
 - Sparsity
 - Popularity bias
 - Long-tail items
 - Cold-start users/items
- Use:
 - Faker
 - Controlled probability distributions
 - Business rules (e.g., user preferences by category)

Deliverable:

Synthetic Data Design & Validation Report

Synthetic Data Validation & EDA

Students must:

- Validate realism of generated data
- Analyze sparsity & interaction density
- Show popularity distribution
- Compare synthetic data properties with real-world expectations

 Mandatory plots:

- User activity distribution
- Item popularity distribution
- Interaction matrix sparsity

Baseline Recommendation Models (ML)

Mandatory Baselines

- Popularity-based recommender
- User-based Collaborative Filtering
- Item-based Collaborative Filtering
- Matrix Factorization (SVD / ALS)

 Implement using **Scikit-learn** / **Surprise**

Content-Based Recommendation (NLP)

Using synthetic item metadata:

- TF-IDF on item descriptions
- Cosine similarity-based recommendations
- Cold-start item handling

Deep Learning Recommendation Model (PyTorch)

Mandatory: Neural Collaborative Filtering (NCF)

Requirements:

- User embeddings
- Item embeddings
- Fully connected layers
- Regularization & early stopping
- Implicit or explicit feedback loss

 Compare:

- Classical CF vs NCF
- Performance vs scalability

Hybrid Recommendation System (ADVANCED)

Combine:

- Collaborative Filtering
- Content-Based similarity
- Deep Learning scores

 Students must design:

- Score fusion strategy
- Cold-start fallback logic

Recommendation Evaluation (CRITICAL)

Mandatory Metrics

- Precision@K
- Recall@K
- MAP@K
- NDCG@K

Evaluation constraints:

- Time-based train-test split

- No interaction leakage

 Deliverable:

Evaluation & Benchmarking Report

Explainability Layer

Students must explain:

- Why an item is recommended
- Similar users/items
- Content similarity justification

Application Layer (MANDATORY – Choose ONE)

Option A: FastAPI Recommendation Service

Endpoints:

- `/recommend/{user_id}`
- `/similar-items/{item_id}`

Response:

- Top-N items
- Explanation text

Option B: Streamlit Recommendation Dashboard

UI Features:

- User selection
- Recommended items
- Similar items
- Explanation panel
- Cold-start demo

System Design & Scalability

Students must propose:

- Batch vs real-time recommendation flow
- Retraining strategy
- Data growth handling
- Monitoring KPIs



Deliverable:

Recommendation System Architecture Document

Important Rule

- ✗ Notebook-only solutions will **NOT** be accepted
- ✓ Either **FastAPI OR Streamlit** is mandatory

Technologies and Tools

Layer	Technology
Synthetic Data	Faker, NumPy
Data Processing	Pandas
ML Recommenders	Scikit-learn, Surprise
NLP	TF-IDF
Deep Learning	PyTorch
Evaluation	Custom metrics
API / UI	FastAPI / Streamlit
Version Control	Git
Deployment (Optional)	Docker

Project Deliverables

data/
synthetic_data/
notebooks/
src/
models/
app/
reports

Mandatory Notebooks

- Advanced EDA
- Feature Engineering
- ML Pipeline
- Deep Learning Model
- Explainability & Governance

Reports

- Data Risk Assessment
- Cost Optimization
- Governance & Fairness
- Final Business Recommendations

Project Evaluation Metrics

Criteria	Marks
Synthetic Data Design & Realism	15
Baseline Recommenders	10
Deep Learning Model (PyTorch)	20
Hybrid Recommendation Logic	15
Evaluation Metrics	15
Explainability	10
Total	85

Application Layer Evaluation (15 Marks)

FastAPI Evaluation

Criteria	Marks
API Design & Correctness	5
Input Validation & Pipeline Integration	5
Business-Meaningful Response	5
Total	15

Streamlit Evaluation

Criteria	Marks
UI Usability & Clarity	5
Explainability Visualization	5
Business Recommendation Display	5
Total	15

Bonus (+10 Marks)

- Dockerized API / App
- Batch prediction support
- Executive summary dashboard
- Model monitoring proposal

Project Guidelines:

1. GitHub Repository

- Push your final project to your GitHub account.
- Repository must be private during the assignment period.
- Add the instructor's GitHub ID as a collaborator for review.

2. Naming Convention

- GitHub Repo Name: deep-learning-recommendation-system-`{yourname}`
- Example: deep-learning-recommendation-system-`amit`

Timeline:

Define the project timeline, including milestones and deadlines.

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.

Note: Book the slot at least before 12:00 Pm on the same day

Timing: Monday-Saturday (4:00PM to 5:00PM)

Booking link : <https://forms.gle/XC553oSbMJ2Gcfug9>

For DE/BADM project/class topic doubt slot clarification session:

Booking link : <https://forms.gle/NtkQ4UV9cBV7Ac3C8>

Session timing:

For DE: 04:00 pm to 5:00 pm every saturday

For BADM 05:00 to 07:00 pm every saturday

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.

Note: This form will Open only on Saturday (after 2 PM) and Sunday on Every Week

Timing:

For BADM and DE

Monday-Saturday (11:30AM to 1:00PM)

For DS and AIML

Monday-Saturday (05:30PM to 07:00PM)

Booking link : <https://forms.gle/1m2Gsro41fLtZurRA>